Claude Code i Cat Goes Home

Översikt över projektets struktur och vad varje del gör

Alla delar har ett kort startinnehåll. Tanken är att först visa strukturen och sedan skriva det riktiga innehållet fil för fil.

Filstruktur

```
the_cat_game/
    CLAUDE.md                          projektinstruktioner
    CLAUDE.local.md                    dina personliga anteckningar (committas inte)
    .mcp.json                          MCP-servrar
    .claude/
        settings.json                  behörigheter och hooks
        settings.local.json            dina personliga inställningar (committas inte)
        commands/smoke-test.md         /smoke-test
        skills/add-obstacle/SKILL.md   /add-obstacle
        agents/riddle-reviewer.md      subagent
        rules/riddles.md               regel för gåtfiler
        hooks/check_python_syntax.py   hook-skript
        output-styles/game-mentor.md   svarsstil
```

Vad varje del gör

Del	Vad den är till för	När den gäller	Startinnehåll
`CLAUDE.md`	Projektets bakgrund och konventioner	Varje session	Projektets upplägg, hur det körs och två konventioner
`CLAUDE.local.md`	Din personliga version av CLAUDE.md	Varje session, bara för dig	Platshållare
`settings.json`	Vad Claude får göra utan att fråga och vad som är blockerat	Alltid	Tillåter git-kommandon som bara läser och blockerar läsning av .env-filer
`settings.local.json`	Dina personliga inställningar	Alltid, bara för dig	Tom behörighetslista och vald svarsstil
`smoke-test.md`	En sparad prompt	När du skriver /smoke-test	Startar main.py och avslutar direkt
`add-obstacle`	En instruktion som också kan innehålla extra filer	När du skriver /add-obstacle eller när Claude ser att den passar	Steg för att lägga till ett hinder
`riddle-reviewer.md`	En hjälpagent med egen kontext och begränsade verktyg	När Claude delegerar till den eller du ber om den	Granskar gåtorna utan att ändra något
`riddles.md`	Instruktioner för vissa filer	När Claude arbetar med matchande filer	Gåtornas format
`check_python_syntax.py`	Ett skript som körs automatiskt, så att det inte hänger på att Claude kommer ihåg	Efter att Claude skrivit eller ändrat en fil	Rapporterar Python-syntaxfel till Claude
`game-mentor.md`	Ändrar hur Claude svarar	Bara när du väljer den	Förklarar resonemanget bakom varje ändring
`.mcp.json`	Kopplar Claude till externa verktyg	När du har godkänt den	Dina Phoenix-traces

Bra att veta

- **Två skills-mappar:** `skills/` är spelinnehåll för ADK-agenterna och `.claude/skills/` är för Claude Code.
- **Claude-verktyg:** det finns ingen fil eller mapp för verktyg. De är antingen inbyggda eller kommer från MCP. Du styr dem med behörigheter i `settings.json` och med raden `tools:` i en subagentfil.
- **Kommandon och skills:** båda ger en genväg med `/namn`. Använd ett kommando för en kort sparad prompt och en skill för större uppgifter eller sådant som Claude ska kunna använda på egen hand.
- **Minne:** ingår inte i strukturen, eftersom Claude sköter det själv utanför projektmappen.
- **Plugins:** ingår inte, eftersom de är till för att dela en uppsättning mellan projekt.

Innan du testar

- **Phoenix:** MCP-servern kräver Node.js och har inget att läsa om inte `phoenix serve` körs.
- **/smoke-test:** gör ett riktigt modellanrop, så API-nyckeln måste finnas i `cat_goes_home/.env`. Kommandot förutsätter också att `python` pekar på projektets miljö.

Claude Code (slash-kommandon)

Dessa kommandon körs inte i bash, utan skrivs vid prompten inuti en Claude Code-session (startad med `claude`).

Vanliga inbyggda kommandon

Kommando	Beskrivning
`/help`	Visar hjälp och tillgängliga kommandon.
`/init`	Initierar projektet med en `CLAUDE.md`-guide.
`/memory`	Redigerar `CLAUDE.md`-filer och visar/aktiverar auto-memory.
`/clear`	Startar en ny konversation med tom kontext (alias: `/reset`, `/new`).
`/compact [instruktioner]`	Sammanfattar konversationen för att frigöra kontext.
`/context [all]`	Visualiserar aktuell kontextanvändning.
`/model [modell]`	Byter AI-modell för sessionen/som standard.
`/effort [nivå]`	Sätter ansträngningsnivå (`low`–`xhigh`, `max`, `auto`).
`/permissions`	Hanterar allow/ask/deny-regler för verktygsbehörigheter (alias: `/allowed-tools`).
`/mcp`	Hanterar MCP-serveranslutningar.
`/agents`	Påminnelse om att skapa/hantera subagenter (se nedan).
`/plan [beskrivning]`	Går in i plan mode.
`/diff`	Visar ändringar i arbetskatalogen, inklusive Claudes egna.
`/code-review` / `/review`	Granskar aktuell diff eller ett PR-nummer för buggar (`--fix` applicerar fynden).
`/security-review`	Granskar branch-diffen för säkerhetsproblem.
`/rewind`	Återställer konversation och/eller kod till en tidigare punkt (alias: `/checkpoint`, `/undo`).
`/resume [session]`	Återupptar en tidigare konversation (alias: `/continue`).
`/tasks`	Visar och hanterar bakgrundsarbete, inklusive subagenter.
`/usage`	Visar sessionskostnad, plananvändning och statistik (alias: `/cost`, `/stats`).

Kommando	Beskrivning
`/doctor`	Kör en diagnos av installation/konfiguration och föreslår fixar (alias: `/checkup`).
`/status`	Visar version, modell, konto och anslutningsstatus.
`/config`	Öppnar inställningar (tema, modell, output style) (alias: `/settings`).
`/skills`	Listar tillgängliga skills.
`/export [filnamn]`	Exporterar konversationen som text.
`/bug [rapport]`	Rapporterar en bugg med sessionskontext (alias: `/share`).

Subagenter

Subagenter är specialiserade assistenter som körs i egen kontext. De definieras som Markdown-filer med YAML-frontmatter.

Plats	Omfattning
`.claude/agents/<namn>.md`	Gäller bara det aktuella projektet (checka in i Git för att dela med teamet).
`~/.claude/agents/<namn>.md`	Gäller alla dina projekt på datorn.

Exempelfil

```
---
name: code-reviewer
description: Granskar kod för kvalitet och best practices
tools: Read, Grep, Glob
model: sonnet
---

Du är en kodgranskare. Förklara varje problem, visa nuvarande kod
och ge ett förbättrat förslag.
```

Sätt att anropa en subagent i en session

Syntax	Effekt
`Använd code-reviewer-subagenten för att granska mina ändringar`	Naturligt språk — Claude väljer att delegera.
`@code-reviewer granska auth-ändringarna`	@-mention — garanterar att just den subagenten körs.
`claude --agent code-reviewer`	Kör hela sessionen som den subagenten.

Inbyggda subagenter: `Explore` (read-only kodsökning), `Plan` (research under plan mode), `general-purpose` (komplexa flerstegsuppgifter).

Skills (egna kommandon)

En skill är en `SKILL.md`-fil (eller en äldre fil i `.claude/commands/`) som blir ett eget `/kommando`:

```
---
name: deploy
description: Deploy applikationen till produktion
disable-model-invocation: true
---

Deploya $ARGUMENTS till produktion:
1. Kör testsviten
2. Bygg applikationen
3. Pusha till deployment-målet
```

Plats	Omfattning
`.claude/skills/<namn>/SKILL.md`	Projektet (checka in i Git).
`~/.claude/skills/<namn>/SKILL.md`	Alla dina projekt.

Kör skillen med `/deploy`. `disable-model-invocation: true` gör att bara du kan starta den manuellt, inte Claude själv.

Tips: håll alltid `.gitignore` *och* `.dockerignore` *uppdaterade så att virtuella miljöer,* `.env`*-filer och andra lokala/känsliga filer aldrig hamnar i git-historik eller Docker-images av misstag.*